

A. Sichuan Provincial Contest

你准备参加 SCCPC 了。

你发现了一棵有 n 个节点的树，每个节点上都挂着一个大写英文字母。我们用 S_i 来表示第 i 个节点上的字母。

你想知道，这棵树中有多少条恰好包含五个节点的简单路径 u, v, x, y, z ，它们对应的字母序列 $S_u S_v S_x S_y S_z$ 正好是字符串 SCCPC。

0.1 输入

第一行是一个正整数 T ($1 \leq T \leq 10^4$)，表示测试用例的数量。

每个测试用例的第一行是一个整数 n ($1 \leq n \leq 10^6$)，代表树中节点的个数。

第二行是一个长度为 n 的字符串 S ，只包含大写英文字母，其中第 i 个字符 S_i 就是树中第 i 个节点上的字母。

接下来有 $n - 1$ 行，每行两个整数 x_i, y_i ($1 \leq x_i, y_i \leq n, x_i \neq y_i$)，表示树中节点 x_i 和 y_i 之间有一条边。

保证所有测试用例中 n 的总和不超过 2×10^6 。

0.1 输出

每个测试用例输出一个整数，表示符合条件的简单路径数量，单独一行。

0.1 样例

Input

1	2
2	5
3	SCCPC
4	1 2
5	2 3
6	3 4
7	4 5
8	7
9	SCCPCCC
10	1 2
11	2 3
12	3 4
13	4 5
14	4 6
15	4 7

Fence 1

output

1	1
2	3

Fence 2

B Calendar Cubes

输入文件: `standard input`

输出文件: `standard output`

时间限制: 1 second

内存限制: 1024 megabytes

Colin 正在设计一个使用两个立方体的特殊日历。

每个立方体恰好有 6 个面，每个面上标有一个从 0 到 8 的数字。使用这两个立方体，Colin 想以两位数的格式显示日期。

(图片：两个立方体，前面显示 1 和 6，顶面显示 2 和 7)

例如，上面的插图显示了两个立方体，正面显示整数 16。一个立方体显示数字 1，另一个显示数字 6。



Figure 1

为了以两位数格式显示整数 $d \in [01, 99]$ ，选择一个立方体显示十位数字（对于 01 到 09，十位数字是 0），另一个显示个位数字。请注意，数字 6 也可以表示 9。也就是说，如果一个立方体有一个 6，它可以被用作 6 或 9。

给定两个立方体，我们将其 MEX 定义为无法显示的最小正整数（两位数格式）。可以证明，没有两个立方体可以显示从 01 到 99 的所有整数（包括两端）。

给定一个整数 x （以两位数格式给出），你需要构造两个立方体，使得它们的 MEX 恰好是 x 。

1. 输入

有多个测试用例。输入的第一行包含一个整数 T ($1 \leq T \leq 99$)，表示测试用例的数量。对于每个测试用例：

唯一的一行包含一个以两位数格式表示的整数 x ($01 \leq x \leq 99$)。

2. 输出

对于每个测试用例：

- 如果可以构造两个立方体，使得它们的 MEX 恰好为 x ，则首先在一行中输出 `Yes`。然后输出第二行，包含 12 个整数（从 0 到 8），用空格分隔，表示这两个立方体。前 6 个整数是第一个立方体上的数字，而后 6 个整数是第二个立方体上的数字。
- 否则，如果无法做到，只需在一行中输出 `No`。

3. 样例

3.1 standard input

1	4
2	01
3	02
4	99
5	11

Fence 3

3.2 standard output

Plaintext

1	Yes
2	0 0 0 0 0 0 0 0 0 0 0 0
3	Yes
4	1 1 1 1 1 1 0 0 0 0 0 0
5	No
6	Yes
7	4 0 5 7 6 8 0 2 3 1 0 0

Fence 4

C 排名预测 (Ranking Prediction)

你和你的队友刚刚结束了一场 ICPC 比赛。五个小时的比赛耗尽了你的精力，而且你的队友还吃了你的午餐。现在你只能靠在桌子上看着排行榜。颁奖典礼还没有举行，所以排行榜仍然处于封榜状态，这意味着你知道你们队伍在整个比赛期间的提交时间和是否通过，但对于其他队伍，你知道他们每次提交的时间，并且你知道在封榜前每次提交是否被接受（通过），但你不知道封榜后进行的提交是否被接受。

当你查看排行榜时，你注意到了一支你非常关注的队伍。你知道他们在封榜前每次提交的时间和状态，以及封榜后每次提交的时间。你想知道他们的队伍排名是否会**严格高于**你们队。为了确定他们排名严格高于你们的可能性，你还想知道他们在封榜后至少需要解决多少个问题。

在 ICPC 比赛中，罚时根据以下规则计算。假设一支队伍**解决**了 m 个问题，编号从 1 到 m 。对于每个解决的问题 i ，设第一次被接受的提交的时间为 t_i ，在解决该问题之前的提交次数为 c_i 。罚时 p 计算如下：

$$p = \sum_{i=1}^m t_i + 20 \cdot c_i$$

在本题中，不考虑编译错误等不计入罚时的特殊因素。

对于两支队伍 A 和 B ，当且仅当 A 解决的问题数量大于 B 解决的数量，或者 A 和 B 解决的问题数量相等且 A 的罚时少于 B 的罚时，队伍 A 被认为排名严格高于队伍 B 。

0.3 输入

第一行包含一个整数 T ($1 \leq T \leq 100$)，表示测试用例的数量。

对于每个测试用例，第一行包含三个整数 n, a, b ($10 \leq n \leq 15, 1 \leq a \leq n, 0 \leq b \leq 10^5$)，表示比赛中有 n 个问题，你的队伍在比赛结束时解决了 a 个问题，罚时为 b 。

第二行包含一个整数 s ($0 \leq s \leq 10^3$)，表示你关注的队伍在比赛期间的提交总次数。

接下来的 s 行，每行包含一个整数和两个字符串 t, p, v ($0 \leq t < 300$)。这表示在第 t 分钟，对问题 p 进行了一次提交，结果是 v 。保证提交按时间顺序给出（这意味着当 t 相同时，提交顺序即为输入的顺序）， p 是前 n 个大写字母之一，且 $v \in \{\text{ac}, \text{rj}, \text{pd}\}$ 。**ac** 表示该提交被接受，**rj** 表示该提交被拒绝，**pd** 表示该提交处于封榜状态，未知该提交是否被接受。保证当 $t < 240$ 时， v 不是 **pd**；当 $t \geq 240$ 时， v 保证是 **pd**。

0.4 输出

对于每个测试用例，在新的一行输出一个整数。如果你关注的队伍最终排名不可能严格高于你的队伍，输出 -1 ；否则，输出为了使他们的最终排名严格高于你的队伍，他们在封榜后至少需要解决的问题数量。

0.5 样例

Input

```
1 1
2 11 6 900
3 13
4 11 C ac
5 34 J ac
6 52 D rj
7 61 D ac
8 193 A rj
```

9	207	A	rj
10	220	G	ac
11	245	A	pd
12	247	A	pd
13	262	H	pd
14	299	A	pd
15	299	C	pd
16	299	K	pd

Fence 5

Output

Plaintext

1	2
---	---

Fence 6

D. Critical Strike

输入文件: `standard input`

输出文件: `standard output`

时间限制: 1 second

内存限制: 1024 megabytes

在一款流行的动作游戏中，商店里有 n 种不同的暴击装备可供选择。第 i 件装备有 $\frac{p_i}{100}$ 的概率触发暴击，暴击倍数为 v_i ，价格为 w_i 枚金币。每件装备只能购买一次。

当多件装备同时触发暴击时，只有最高倍数生效。如果没有装备触发暴击，则有效暴击倍数视为 0。

你最多可以花费 m 枚金币来购买一些装备。你需要最大化你所购买装备的期望有效暴击倍数。

1. 输入

包含多个测试用例。输入的第一行包含一个整数 T ($1 \leq T \leq 300$)，表示测试用例的数量。对于每个测试用例：

第一行包含两个整数 n 和 m ($1 \leq n, m \leq 3 \times 10^3$)，表示装备的数量和你可以花费的最大金币数。

接下来的 n 行中，第 i 行包含三个整数 p_i, v_i 和 w_i ($1 \leq p_i \leq 100, 1 \leq v_i \leq 10^9, 1 \leq w_i \leq 3 \times 10^3$)，表示第 i 件装备的属性。

保证所有测试用例中 n 的总和与 m 的总和均不超过 3×10^3 。

2. 输出

对于每个测试用例，输出一行，包含一个实数，表示最大的期望有效暴击倍数。

如果你的答案的绝对或相对误差不超过 10^{-6} ，则你的答案将被视为正确。形式上，假设你的答案是 a ，裁判的答案是 b ，当且仅当 $\frac{|a-b|}{\max(1, |b|)} \leq 10^{-6}$ 时，你的答案会被接受。

3. 样例

3.1 standard input

```
1 2
2 3 15
3 75 50 5
4 50 100 10
5 25 200 15
6 2 10
7 50 10 100
8 70 30 1000
```

Fence 7

3.2 standard output

Plaintext

```
1 68.750000000000
2 0.000000000000
```

Fence 8

E. 最小生成树 (Minimum Spanning Tree)

给定一个有 n 个顶点和 m 条带权边的无向连通图，你最多可以向图中再添加 k 条边。如果你添加的边连接了顶点 u 和 v ，那么它的权重将为 $|u - v|$ 。

最小化最小生成树中边的总权重。

请注意，图可能包含自环或重边，并且当两个顶点之间已经存在一条或多条边时，允许在它们之间再添加边。

0.3 输入

有多个测试用例。输入的第一行包含一个整数 T ($1 \leq T \leq 10^4$)，表示测试用例的数量。对于每个测试用例：

第一行包含三个整数 n 、 m 和 k ($2 \leq n \leq 2 \times 10^5, n - 1 \leq m \leq 2 \times 10^5, 0 \leq k \leq 2 \times 10^5$)，分别表示图中的顶点数、图中的边数以及你可以添加的边数。

在接下来的 m 行中，第 i 行包含三个整数 u_i 、 v_i 和 w_i ($1 \leq u_i, v_i \leq n, 0 \leq w_i \leq 10^9$)，表示连接顶点 u_i 和 v_i 且权重为 w_i 的一条边。

保证给定的图是连通的。同时保证所有测试用例中 n 的总和、 m 的总和以及 k 的总和均不超过 2×10^5 。

0.4 输出

对于每个测试用例：

首先，输出一行包含一个整数 c ($0 \leq c \leq k$)，表示你要添加的边的数量。

然后输出 c 行，其中第 i 行包含两个整数 u_i 和 v_i ($1 \leq u_i, v_i \leq n$)，表示你添加的第 i 条边连接顶点 u_i 和 v_i ，且其权重为 $|u_i - v_i|$ 。

接着输出一行包含一个整数，表示最小生成树的最小总权重。

最后输出一行，包含 $(n - 1)$ 个不同的整数 $e_1, e_2, \dots, e_{n-1}$ ($1 \leq e_i \leq m + c$)，用空格分隔，表示最小生成树上的边的索引。请注意， $1 \leq e_i \leq m$ 表示原始图中的边，而 $e_i > m$ 表示你添加的第 $(e_i - m)$ 条边。

0.5 样例

Input

```
1 3
2 5 6 2
3 1 2 3
4 2 3 5
5 1 4 7
6 4 2 4
7 5 4 8
8 3 5 1
9 4 5 100
10 1 2 2
11 2 3 0
12 2 4 0
13 4 1 1
```

14	3 4 3
15	3 2 0
16	1 2 100
17	2 3 100

Fence 9

Output

Plaintext

1	2
2	2 3
3	3 4
4	6
5	8 1 6 7
6	0
7	1
8	2 3 4
9	0
10	200
11	1 2

Fence 10

F. 序列操作 (Sequence Operation)

输入文件: `standard input`

输出文件: `standard output`

时间限制: 1 second

内存限制: 1024 megabytes

小青鱼 (Little Cyan Fish) 手里有一个长度为 n 的非负整数序列 $a_1, a_2, \dots, a_n$ 。小青鱼可以执行以下操作任意次数:

- 选择一个整数 $1 \leq i \leq n$ 。
- 将 a_i 更改为 $\text{mex}_{1 \leq k < i} a_k$ 。

这里, 对于一个非负整数集合 S , 符号 $\text{mex } S$ 被定义为未出现在集合 S 中的最小非负整数。例如, $\text{mex}\{0, 1, 2, 4, 6\} = 3$, $\text{mex}\{1, 2, 3\} = 0$, $\text{mex } \emptyset = 0$ 。

小青鱼希望通过若干次操作 (可能为 0 次) 使 a_n 的值尽可能大。小青鱼想让你计算:

1. 在进行任意次数的操作后, a_n 的最大可能值是多少;
2. 在满足 1 的基础上 (即达到最大值), 实现该最大值所需的最少操作次数是多少。

1. 输入

包含多个测试用例。输入的第一行包含一个整数 T ($1 \leq T$), 表示测试用例的数量。

对于每个测试用例, 第一行包含一个整数 n ($1 \leq n \leq 10^6$), 表示序列的长度。

下一行包含 n 个整数 $a_1, a_2, \dots, a_n$ ($0 \leq a_i \leq 10^9$)。

保证所有测试用例中 n 的总和不超过 10^6 。

2. 输出

对于每个测试用例, 输出一行, 包含两个整数。第一个整数表示 a_n 的最大值, 第二个整数表示所需的最少操作次数。

3. 样例

3.1 standard input

```
1 4
2 2
3 0 114514
4 3
5 0 1 0
6 4
7 3 1 2 0
8 6
9 6 5 4 3 2 1
```

3.2 standard output

Plaintext

1	114514 0
2	2 1
3	3 2
4	5 3

Fence 12

G. 航班追踪器 (Flight Tracker)

假设地球是三维欧几里得空间中一个以 $(0, 0, 0)$ 为中心、半径为 r 的球体。有一架航班沿着地球表面从出发地到目的地的最短路径飞行。

作为一名航空爱好者，你拥有一台接收器，它可以接收到距离不超过 d 的航班发出的信号。请注意，我们通过测量地球表面上的最短路径来计算两点之间的距离，这**不是**三维欧几里得空间中的直线距离。你需要找到最小的 d ，以便你的接收器在某个时刻能够接收到航班的信号。

0.3 输入

包含多个测试用例。输入的第一行包含一个整数 T ($1 \leq T \leq 10^4$)，表示测试用例的数量。对于每个测试用例：

第一行包含一个整数 r ($1 \leq r \leq 100$)，表示地球的半径。

第二行包含三个整数 a 、 b 和 c ($-100 \leq a, b, c \leq 100, a^2 + b^2 + c^2 > 0$)，表示你所在位置的坐标为 $\left(\frac{ra}{\sqrt{a^2+b^2+c^2}}, \frac{rb}{\sqrt{a^2+b^2+c^2}}, \frac{rc}{\sqrt{a^2+b^2+c^2}}\right)$ 。

第三行包含三个整数 u 、 v 和 w ($-100 \leq u, v, w \leq 100, u^2 + v^2 + w^2 > 0$)，表示出发地的坐标为 $\left(\frac{ru}{\sqrt{u^2+v^2+w^2}}, \frac{rv}{\sqrt{u^2+v^2+w^2}}, \frac{rw}{\sqrt{u^2+v^2+w^2}}\right)$ 。

第四行包含三个整数 x 、 y 和 z ($-100 \leq x, y, z \leq 100, x^2 + y^2 + z^2 > 0$)，表示目的地的坐标为 $\left(\frac{rx}{\sqrt{x^2+y^2+z^2}}, \frac{ry}{\sqrt{x^2+y^2+z^2}}, \frac{rz}{\sqrt{x^2+y^2+z^2}}\right)$ 。

保证出发地和目的地不会在地球上重合，也不会处于对跖点（完全相对）的位置。因此，地球表面上从出发地到目的地的最短路径是唯一确定的。

0.4 输出

对于每个测试用例，输出一行，包含一个实数，表示最小的 d ，以便你的接收器能在某个时刻接收到航班的信号。

如果你的答案的绝对误差或相对误差不超过 10^{-4} ，则视为正确。形式上来说，假设你的输出是 a ，裁判的答案是 b ，当且仅当 $\frac{|a-b|}{\max(1, |b|)} \leq 10^{-4}$ 时，你的输出才会被接受。

0.5 样例

Input

```
1 2
2 100
3 1 1 1
4 1 0 0
5 0 1 0
6 100
7 -1 -1 0
8 1 0 0
9 0 1 0
```

Fence 13

Output

Plaintext

1	61.547970867038734110
2	235.619449019234492887

Fence 14

0.6 提示

配图展示了第一个样例，其中 P 是你所在的位置， S 是出发地， T 是目的地。

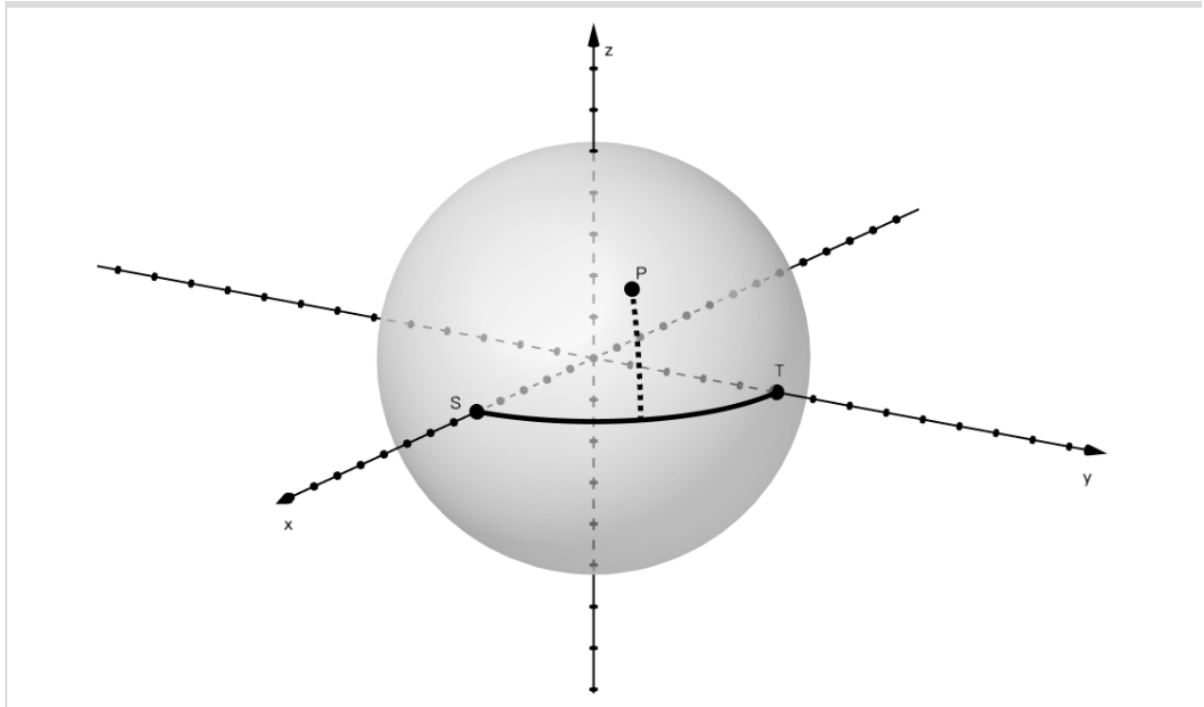


Figure 2

H. Puzzle

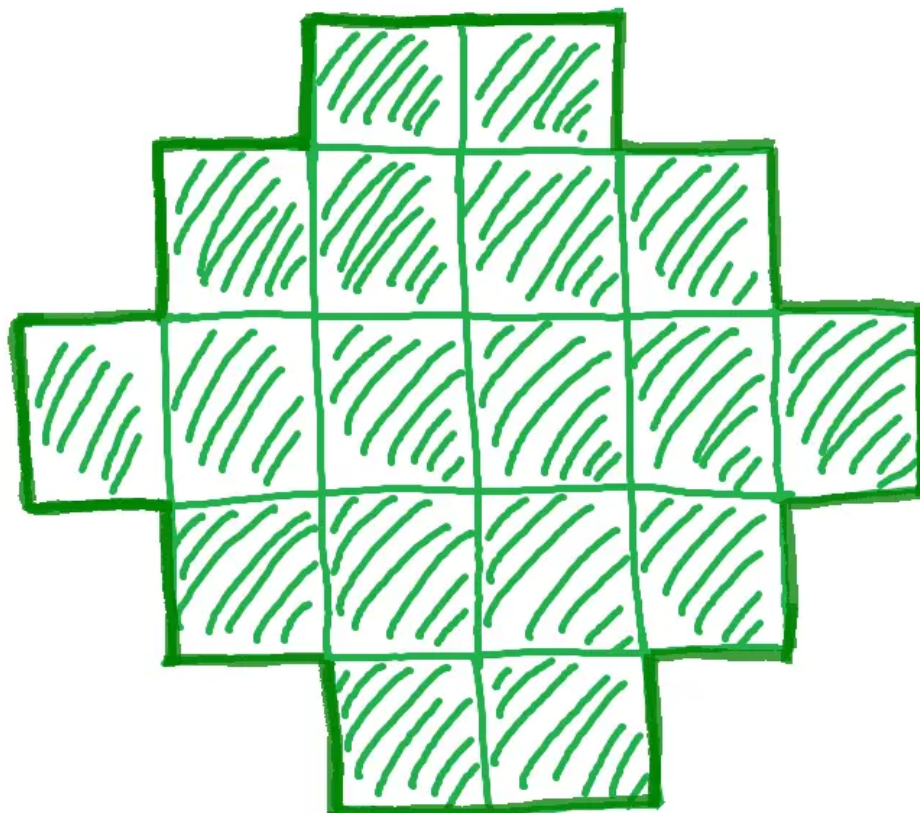
1. 题目描述

你收到了一份拼图礼物，每块拼图都是边长为 1 的正方形。你可以在这幅拼图上粘贴任意图片，然后将其切割，得到一副几乎普通的拼图。

你的朋友是一位狂热的数学家，于是他建议你考虑如下问题。是否可以将这些拼图块排列，使得满足以下条件：

- 拼图块的边与坐标轴平行；
- 拼图块之间不重叠；
- 所有拼图块组成一个连通块（即任意两块之间存在一条路径，每一步都经过相邻且共边的拼图块）；
- 该连通块的周长与面积之比等于 $\frac{p}{s}$ ；
- 使用的拼图块数量不超过 50 000。

你能解决这个问题吗？



对于上图，周长与面积之比为 $\frac{11}{9}$ 。

Figure 3

2. 输入格式

每个测试包含若干组数据。第一行包含一个整数 t ($1 \leq t \leq 10$)，表示测试组数。接下来每组数据一行，包含两个整数 p 和 s ($1 \leq p, s \leq 50$)。

3. 输出格式

对于每组测试数据：

- 如果无法按照上述要求排列拼图块，输出一行 -1 ；
- 否则，第一行输出一个整数 k ($1 \leq k \leq 50\,000$)，接下来 k 行，每行两个整数 x_i 和 y_i ($-10^9 \leq x_i, y_i \leq 10^9$)，表示每块拼图块的坐标。如果存在多种满足条件的拼图方案，输出任意一种即可。

4. 输入输出样例 #1

4.1 输入 #1

1	2
2	1 1
3	31 4

Fence 15

4.2 输出 #1

1	20
2	3 7
3	3 8
4	6 4
5	6 5
6	3 5
7	4 4
8	4 5
9	4 3
10	3 4
11	5 3
12	5 4
13	5 7
14	3 6
15	4 6
16	5 5
17	5 6
18	4 7
19	4 8
20	6 6
21	6 7
22	-1

Fence 16

5. 输入输出样例 #2

5.1 输入 #2

1	2
2	4 2
3	12 5

Fence 17

5.2 输出 #2

1	24
2	-7 2
3	-3 -3
4	-7 -5
5	-7 1
6	-3 2
7	-7 -2
8	-3 -5
9	-7 -6
10	-5 -6
11	-3 -4
12	-3 -6
13	-7 0
14	-6 -6
15	-7 -3
16	-5 2
17	-7 -1
18	-3 1
19	-4 -6
20	-3 0
21	-7 -4
22	-6 2
23	-4 2
24	-3 -1
25	-3 -2
26	5
27	0 0
28	0 1
29	1 0
30	-1 0
31	0 -1

Fence 18

6. 说明/提示

在第一个测试的第一组数据中，图形可能如下所示：

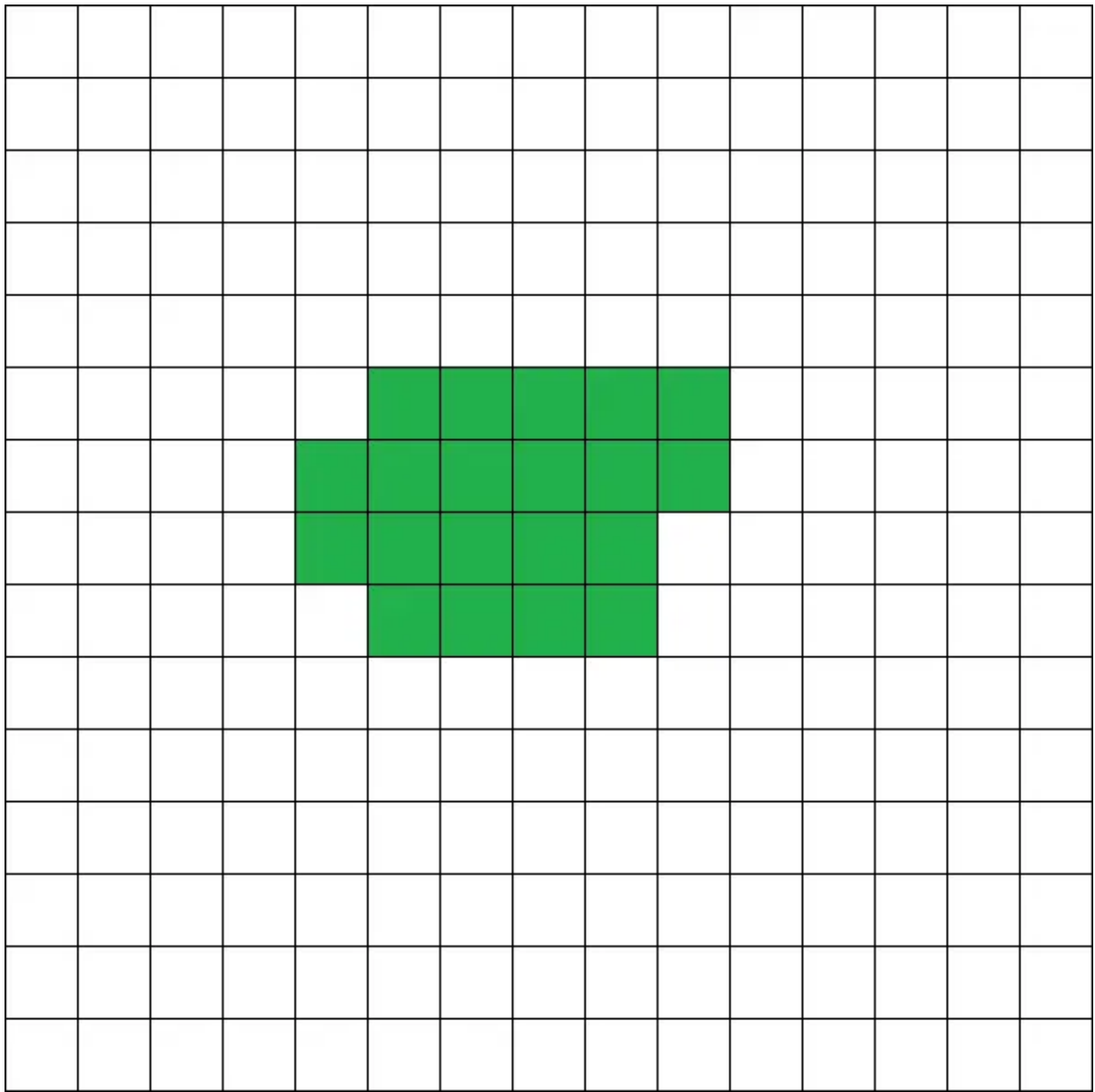
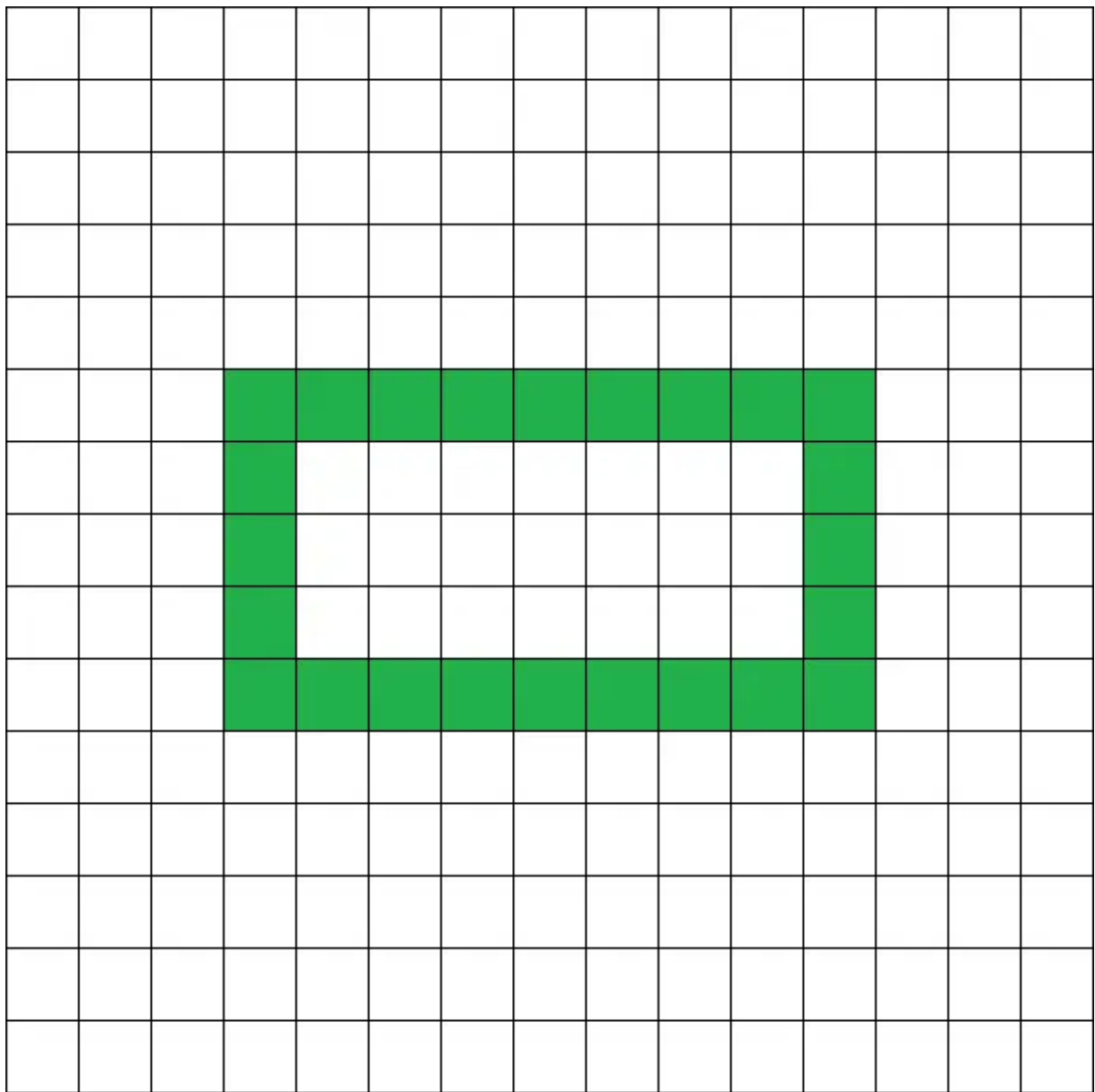


Figure 4

在第二组测试中，图形可能如下：



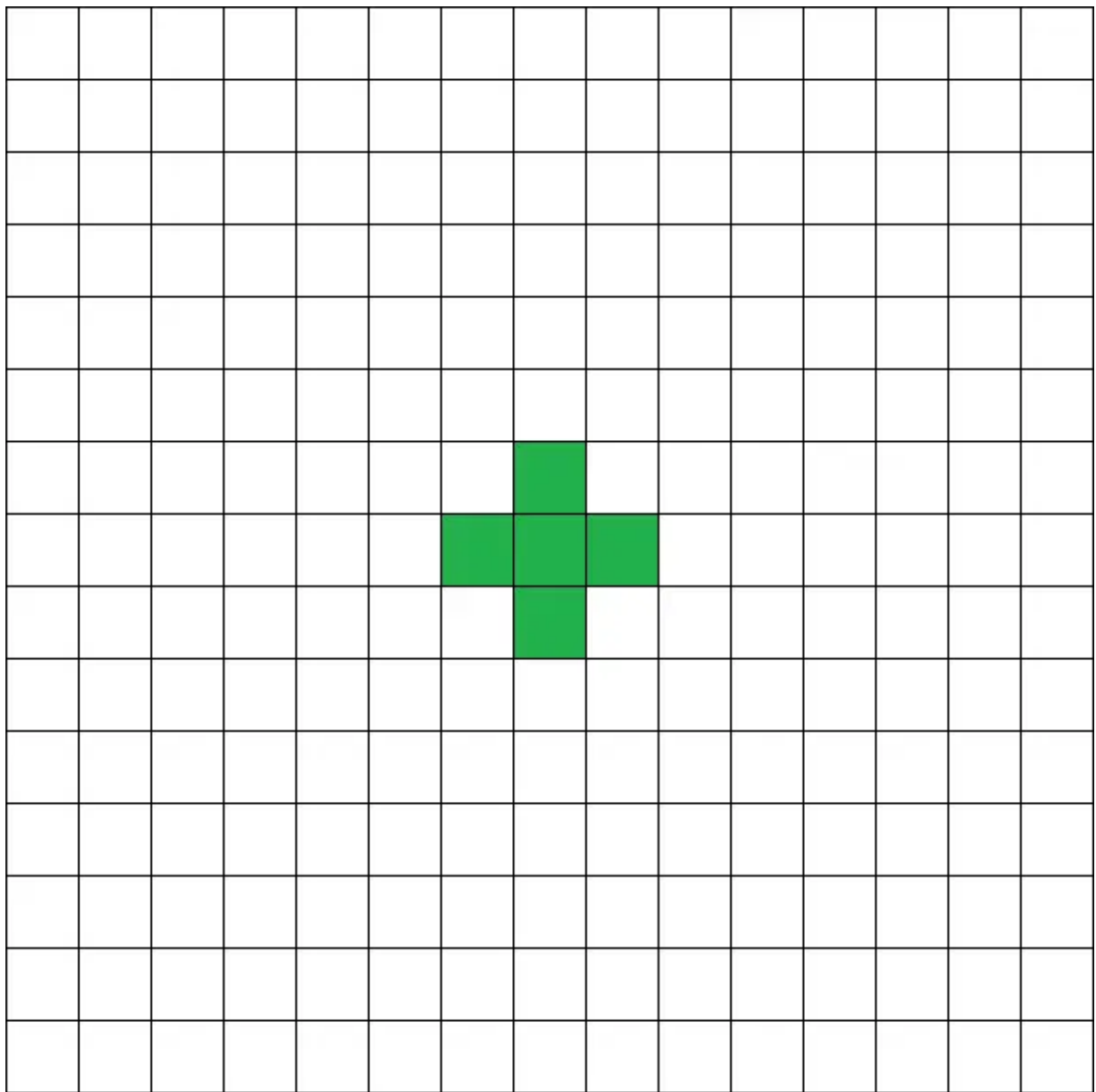


Figure 5

注意，内部的周长也需要计入！

I. 谜题：格利米斯的工匠 (Puzzle: The Artisan of Glimmith)

输入文件：standard input

输出文件：standard output

时间限制：1 second

内存限制：1024 megabytes

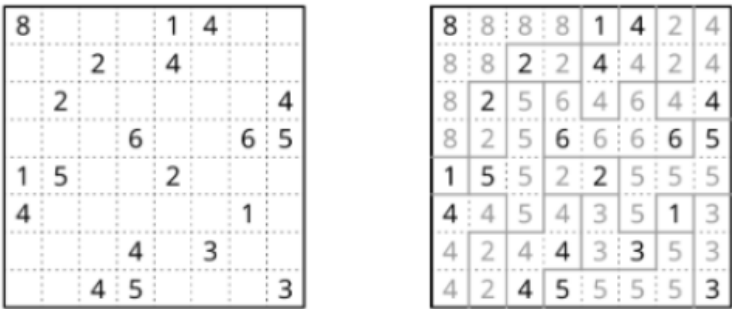
在练习了 Fillomino 的技巧之后，小青鱼（Little Cyan Fish）决定在“The Artisan of Glimmith”游戏中进行更多训练。

小青鱼首先向你介绍了 Fillomino 的规则：

对于一个 m 行 n 列的矩形，将网格划分为若干个区域（每个区域的单元格必须正交连通），使得任意两个相邻的区域具有不同的面积。

网格中的一些单元格已经填入了数字。数字表示该单元格所属区域的面积。

(附图：Fillomino 谜题示例。来源：Puzzle GP 2022 R4)



An example of a Fillomino puzzle. Source: Puzzle GP 2022 R4

Figure 6

现在，小青鱼给了你一个 Fillomino 棋盘，他希望你为这个 Fillomino 谜题构造出任意一个有效的解答。

当然，这个问题对小青鱼来说太难了，所以你只需要解决 $m = 1$ 的情况。换句话说，给定一行 n 个单元格，你想将这些单元格划分成若干个线段（每个线段完全包含几个连续的单元格），要求：

- 任意两个相邻的线段具有不同的长度；
- 某些单元格包含一些数字，这些数字表示该单元格所属线段中的单元格数量（即线段的长度）。

小青鱼要求你判断该棋盘是否存在有效的解答。如果存在，你还需要输出任意一个有效的解答。

1. 输入

包含多个测试用例。输入的第一行包含一个整数 T ($1 \leq T$)，表示测试用例的数量。

对于每个测试用例，第一行包含一个整数 n ($1 \leq n \leq 10^6$)。

下一行包含 n 个整数 $a_1, a_2, \dots, a_n$ ($0 \leq a_i \leq n$)。如果 $a_i = 0$ ，表示第 i 个单元格没有填入数字。否则，表示第 i 个单元格中填入了数字 a_i 。

保证所有测试用例中 n 的总和不超过 10^6 。

2. 输出

对于每个测试用例，如果不存在任何有效的填数方案，则输出单行 **NO**。

否则，输出的第一行包含字符串 `Yes`。下一行输出 n 个整数，表示在你构造的方案中每个单元格所在线段的长度。如果有多个满足要求的方案，你可以输出其中任意一个。

3. 样例

3.1 standard input

```
1 7
2 3
3 0 0 0
4 4
5 2 0 0 0
6 4
7 0 2 0 0
8 6
9 0 3 3 0 3 0
10 10
11 2 0 0 3 0 0 4 0 0 1
12 6
13 0 0 1 1 0 0
14 6
15 1 2 3 4 5 6
```

Fence 19

3.2 standard output

Plaintext

```
1 Yes
2 2 2 1
3 No
4 Yes
5 1 2 2 1
6 No
7 Yes
8 2 2 3 3 3 4 4 4 4 1
9 No
10 No
```

Fence 20

J. 方块谜题 (Square Puzzle)

考虑一个 3 行 3 列的网格。网格的每个单元格中都有一个整数。1 到 9（包含 1 和 9）的每个整数在网格中恰好出现一次。

你可以对网格执行以下三种类型的操作。

操作	描述	示例
右移一行 (Right-shift one row)	选择一行并将其最右边的元素移动到最左边的位置。	右移第三行： 2 9 3 → 2 9 3 6 7 8 → 6 7 8 5 1 4 → 4 5 1
下移一列 (Down-shift one column)	选择一列并将其最底部的元素移动到最顶部的位置。	下移第一列： 2 5 7 → 8 5 7 4 9 1 → 2 9 1 8 6 3 → 4 6 3
顺时针旋转 (Rotate clockwise)	将整个网格顺时针旋转 90 度。	顺时针旋转： 5 7 3 → 8 1 5 1 4 9 → 2 4 7 8 2 6 → 6 9 3

Table 1

给定两个这样的网格，计算将第一个网格转换为第二个网格所需的最小操作数。

0.3 输入

包含多个测试用例。输入的第一行包含一个整数 T ($1 \leq T \leq 2 \times 10^5$)，表示测试用例的数量。对于每个测试用例：

前三行中，第 i 行包含一个字符串 $a_{i,1}a_{i,2}a_{i,3}$ ($1 \leq a_{i,j} \leq 9$)，表示第一个网格的第 i 行。

接下来的三行中，第 i 行包含一个字符串 $b_{i,1}b_{i,2}b_{i,3}$ ($1 \leq b_{i,j} \leq 9$)，表示第二个网格的第 i 行。

0.4 输出

对于每个测试用例输出一行。如果可以将第一个网格转换为第二个网格，则输出一个整数，表示所需的最小操作数；如果无法做到，则输出 -1 。

0.5 样例

Input

1	4
2	293
3	678
4	514
5	624
6	579
7	183
8	624
9	579
10	183
11	293
12	678
13	514
14	123
15	456
16	789
17	321
18	456
19	789
20	123
21	456
22	789
23	123
24	456
25	789

Fence 21

Output

Plaintext

1	3
2	5
3	-1
4	0

Fence 22

0.6 提示 (Note)

对于第一个样例测试用例，如描述中的图例所示，我们可以先**右移第三行**，然后**下移第一列**，最后**顺时针旋转**。